

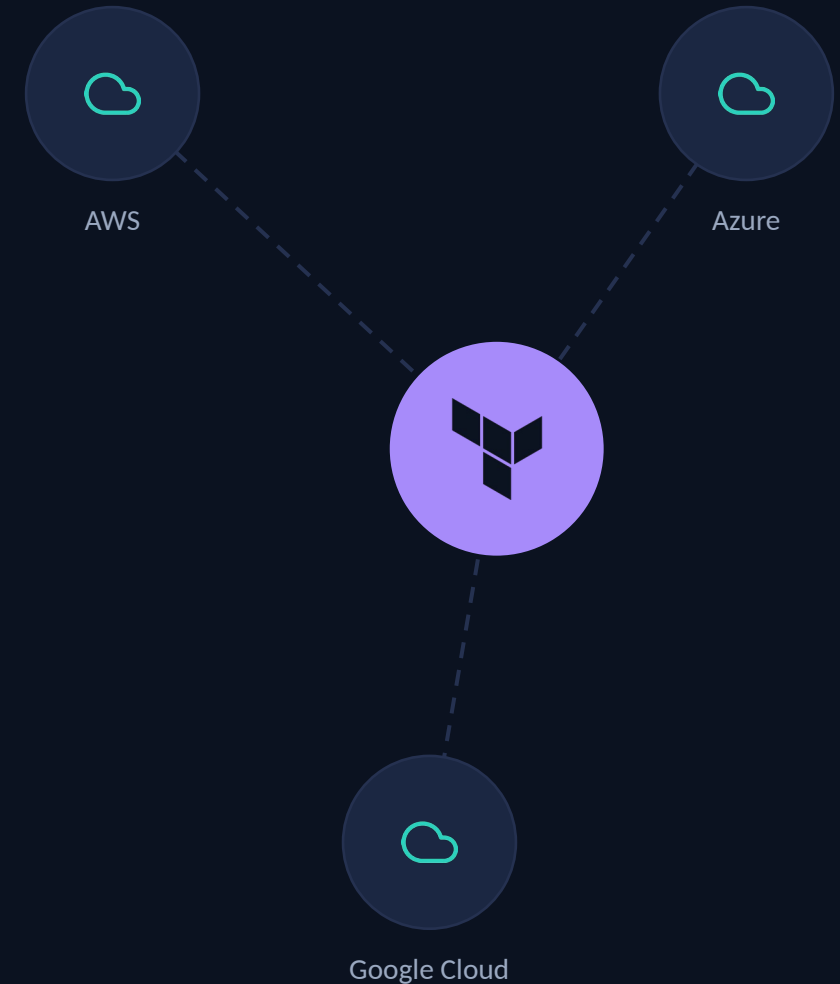
CONF42 DEVSECOPS 2026 • 1 OCTOBER 2026

Terraform for Secure Multi-Cloud DevSecOps

Automating scalable infrastructure as code with security built into every stage: author, plan, apply, operate.

Perumalsamy Ravindran

Staff Software Engineer



About me



Perumalsamy Ravindran

Staff Software Engineer · Infrastructure, cloud platforms, delivery pipelines

- Worked on infrastructure, cloud platforms and delivery pipelines for large enterprises across multiple cloud providers
- Focus: platform engineering, DevSecOps pipelines, governance that developers actually adopt
- This talk: patterns that survived audits, incidents and scale, with sources for every number

What you will leave with

-  A reference architecture for multi-cloud Terraform
-  A pipeline blueprint with security gates at each stage
-  Concrete controls: identity, policy, secrets, state, supply chain
-  An operate loop: drift, testing, observability
-  A 90-day roadmap you can start on Monday

Multi-cloud is the default. Consistency is not.

2.4

public clouds used by the average enterprise; 70% run hybrid

Flexera 2025

82%

rank security a top cloud challenge; cost is #1 at 85%

Flexera 2026

80%

of organizations use at least one IaC tool

Datadog DevSecOps 2025

6%

have their infrastructure estate fully codified

Firefly State of IaC 2025

≥ 38%

still provision production resources by hand in the console ("ClickOps")

Datadog State of DevSecOps 2025

27%

of platform and security teams operate as one unified function

HashiCorp Cloud Complexity Report 2025

The gap is not tooling. It is consistency: several clouds, partial code coverage, and security outside the delivery loop.

Misconfiguration is the threat model

~80%

of documented cloud intrusions in 2025 began with known weaknesses: exposed secrets, misconfigurations, unpatched vulnerabilities

Wiz Cloud Threats Retrospective 2026

40%

of cloud initial access came from exposed secrets (21%) and misconfigurations (19%)

Wiz State of Cloud Risk 2026

59%

of AWS IAM users have an access key older than one year

Datadog State of Cloud Security 2025

28.65M

new hardcoded secrets pushed to public GitHub in 2025 (+34% YoY)

GitGuardian Secrets Sprawl 2026

Every one is a setting. Settings can be declared, reviewed and enforced in code.



Capital One, 2019

SSRF on a misconfigured proxy reached instance credentials for an over-permissive IAM role. ~106M records. \$80M OCC penalty, \$190M settlement.



Microsoft AI research, 2023

One SAS token in a public repo granted full control of a storage account until 2051. 38 TB exposed, including 30,000+ internal messages.

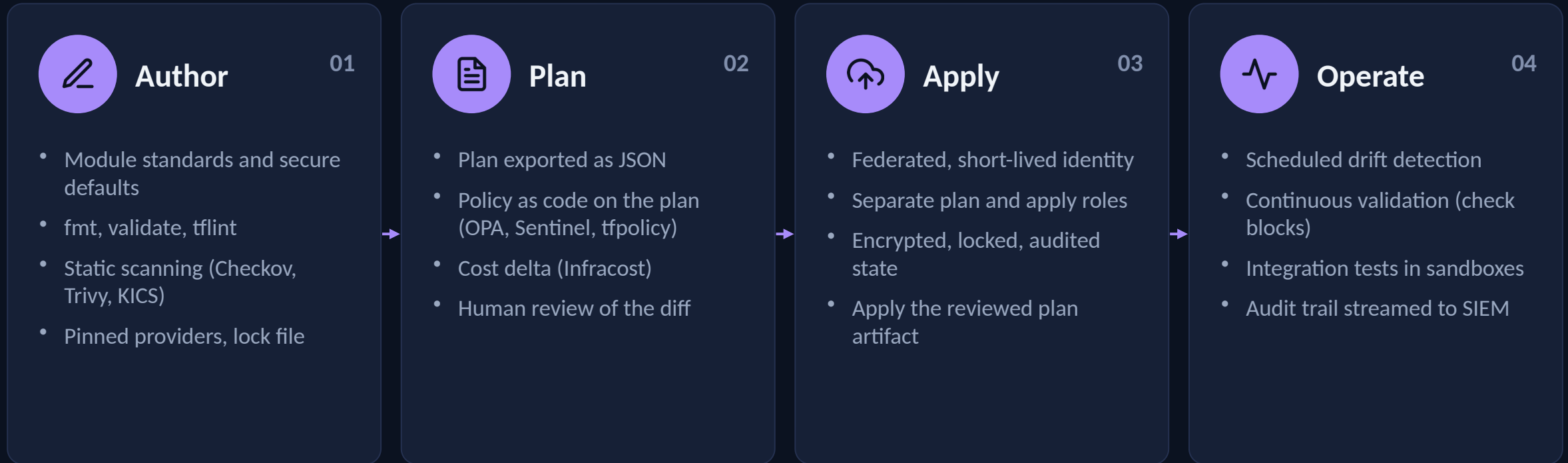


Volkswagen / Cariad, 2024

Misconfigured cloud storage exposed data on ~800,000 EVs for months, with precise location for 460,000 vehicles.

Avg. breach cost 2026: \$4.99M (IBM)

Security is not a stage. It is four stages.



Each stage has a control that fits it naturally. Bolting all of them onto the end is what makes security slow.

What Terraform actually gives you in multi-cloud

One workflow, not one API

Resources stay cloud-specific. What you standardize is everything around them:

HCL language

State model

Plan / apply loop

One pipeline

One policy layer

One registry

7,332 providers · 24,553 modules on the public registry (Sep 2026)



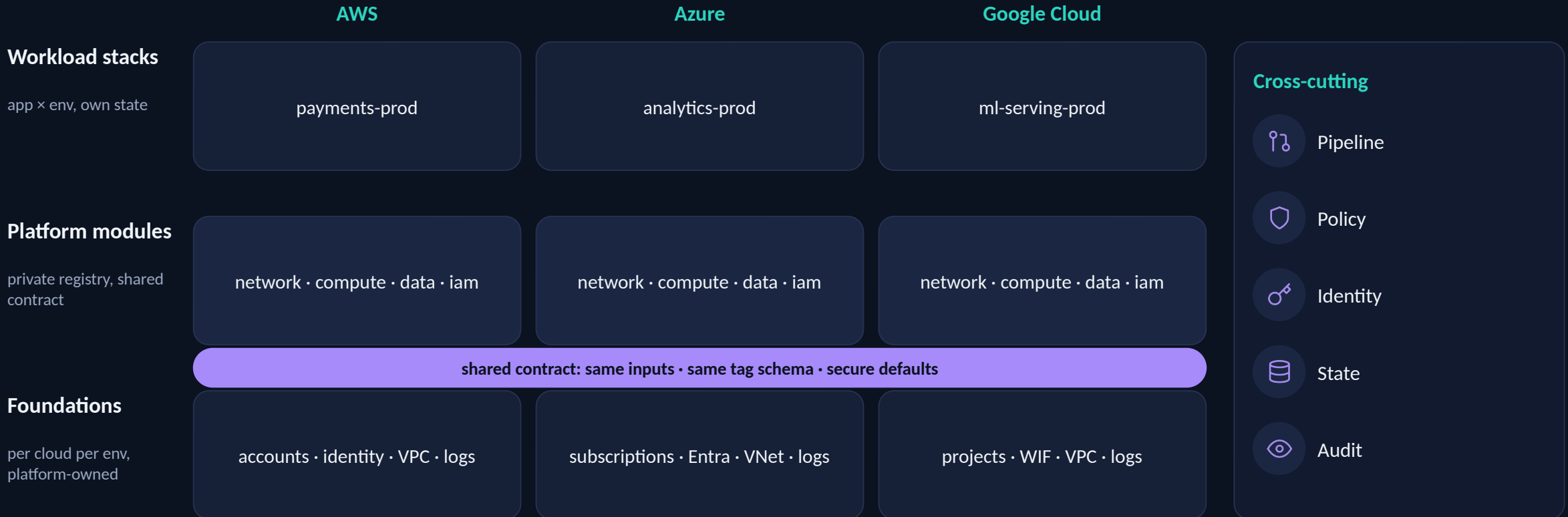
OpenTofu (Linux Foundation, CNCF sandbox)

MPL-licensed fork after the 2023 BSL change. Same workflow, plus client-side state encryption and provider for_each. Every pattern in this talk works on both.

The engine moved: what changed since 2023

- **1.6 / 1.7** Native terraform test with mock providers (Oct 2023, Jan 2024)
- **1.10 / 1.11** Ephemeral values and write-only arguments: secrets never reach plan or state; S3 native locking (Nov 2024, Feb 2025)
- **1.13 / 1.14** Stacks CLI; terraform query and actions (Aug, Nov 2025)
- **1.16 / 1.17 β** Current stable 1.16.4 (Sep 2026); Terraform Policy, a native HCL policy language, in 1.17 beta

Reference architecture: three layers, one contract



- Keep the module tree flat: one level of child modules (HashiCorp guidance)
- Reusable modules never contain provider blocks; providers are passed in
- Per-cloud modules, one interface. Not one "universal" module

Module design rules that scale



1. Secure by default

Encryption on, public access off, logging on. Opting out is an explicit variable that policy can see and reject.



2. Small, validated interface

variable validation blocks; deprecated attribute (1.15) to evolve inputs without breaking consumers.



3. Versioned and pinned

Semantic version tags, a changelog, and consumers pin a version constraint or a commit SHA.



4. Tested

terraform test with mock_provider on every PR (seconds). Terratest in a sandbox for security-boundary modules.



5. Documented and discoverable

terraform-docs for the reference, private registry for discovery, examples/ for the golden path.

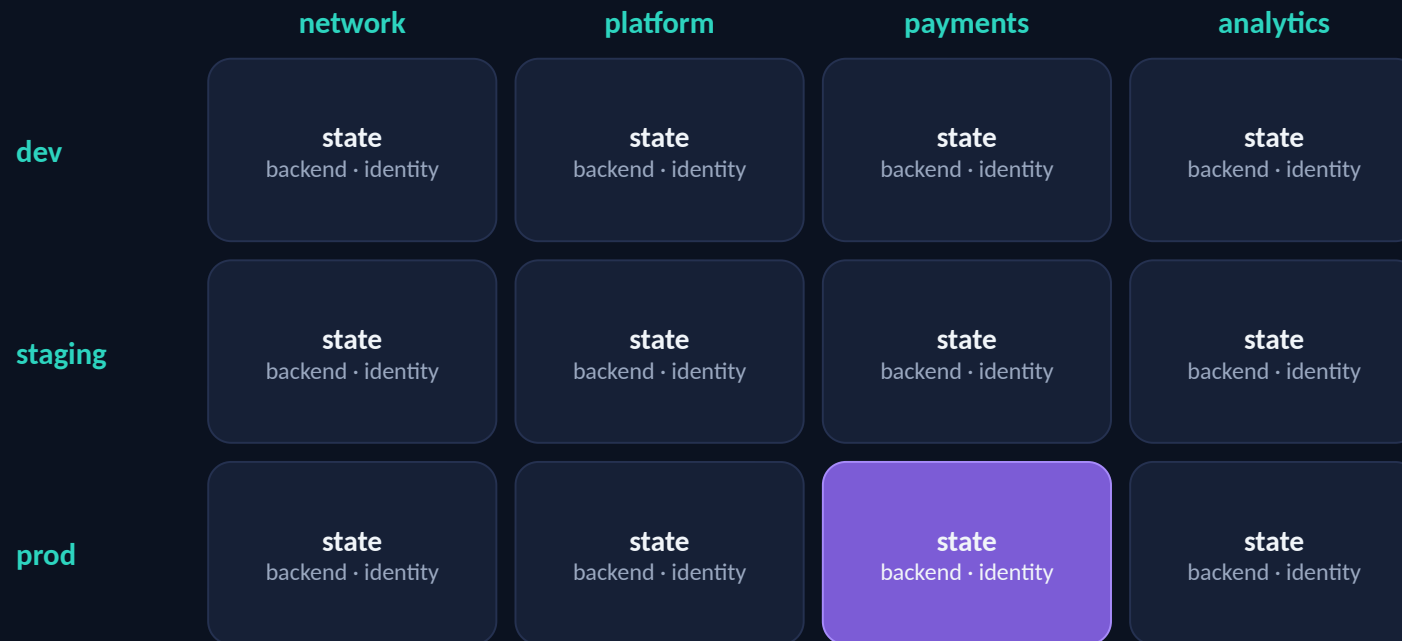


6. Flat composition

No nested module trees, no provider blocks, no environment names or account IDs baked in.

Result: a security review becomes a review of a module version, not of every deployment.

State topology and blast radius



Blast radius of a bad apply = one cell

Separate backend (bucket / storage account) and identity per environment; per-domain keys inside it.

Rules

- Unit of state = one environment × one domain
- Keep states small: < 100 resources, ideally a few dozen (Google Cloud guidance)
- CLI workspaces share a backend and credentials: not an isolation boundary (HashiCorp docs)
- Cross-state reads need the whole file: publish needed outputs to a parameter store instead
- Own the state bucket in a separate, locked-down account per environment

Securing remote state

```
terraform {  
  backend "s3" {  
    bucket      = "org-tfstate-prod"  
    key         = "network/vpc.tfstate"  
    region      = "eu-west-1"  
    encrypt     = true  
    kms_key_id  = "arn:aws:kms:...:key/..."  
    use_lockfile = true    # native lock, TF >= 1.11  
  }  
}  
  
# Azure: use_azuread_auth = true (no account keys)  
# GCS:   kms_encryption_key = "projects/.../cryptoKeys/tf"
```

State is plaintext. Treat the state store as a secrets database.

"Terraform stores your state in a plaintext file, which includes any secret values you defined in your configuration." — HashiCorp docs, *Sensitive data in state*



Encrypt with your own key

SSE-KMS on S3, CMEK on GCS, Entra ID auth on Azure Blob (account keys: "not recommended")



Native locking + versioning

S3 lock file (DynamoDB deprecated), blob leases, GCS. Versioning turns corruption into a rollback



Least-privilege access

Only the plan role, the apply role and an alerting break-glass role can read state



Log every read

A state download is a data exfiltration event. Alert on reads outside the pipeline



Need more?

OpenTofu client-side encryption; HCP Terraform Premium hold-your-own-key

The pipeline blueprint

PLAN PHASE • read-only identity



APPLY PHASE • write identity, protected environment



Same shape on GitHub Actions, GitLab CI, Azure DevOps, HCP Terraform, Spacelift or Atlantis. **Every arrow is an artifact you can audit:** commit SHA, scan report, saved plan, policy verdict, approver, apply log.

Identity: no long-lived keys, split plan from apply

```
# GitHub Actions job (GitLab and Azure DevOps: same idea)
permissions:
  id-token: write      # request an OIDC token
  contents: read
steps:
  - uses: aws-actions/configure-aws-credentials@<sha>
    with:
      role-to-assume: arn:aws:iam::123456789012:role/tf-plan
      role-session-name: gha-${{ github.run_id }}

# IAM trust policy condition (AWS)
"token.actions.githubusercontent.com:sub":
  "repo:acme/infra:environment:prod"
```

"Long-lived credentials are the most common cause of publicly documented cloud security breaches." — Datadog, State of Cloud Security 2024



Federate, never store

CI issues a signed OIDC token per job; AWS, Azure and Google Cloud exchange it for credentials that expire with the job. Nothing to leak or rotate.



Trust conditions do the work

Bind the trust to repository + branch or environment (sub claim). A fork or feature branch cannot assume the prod role.



Two roles: plan and apply

Plan role: read/describe only. Apply role: write, assumable only from the protected environment. HCP Terraform exposes run_phase as a claim.



Name the session after the run

role-session-name = run ID makes every CloudTrail entry traceable to a commit and an approver.

Policy as code: test the plan, not the prose

```
# policy/network.rego (OPA / Conftest, Rego v1)
package terraform.network
import rego.v1

deny contains msg if {
  rc := input.resource_changes[_]
  rc.type == "aws_security_group_rule"
  rc.change.after.type == "ingress"
  "0.0.0.0/0" in rc.change.after.cidr_blocks
  rc.change.after.from_port <= 22
  rc.change.after.to_port >= 22
  msg := sprintf("%s opens SSH to the internet", [rc.address])
}

$ terraform show -json tfplan > plan.json
$ conftest test plan.json -p policy/ --all-namespaces
```

Static scanners read the code. Plan-time policy reads what will actually change, including values computed at plan time. Use both.

Where each tool fits

Static (reads HCL)

Checkov 1,000+ policies · KICS 2,400+ rules · Trivy

Author

Plan-time (reads plan JSON)

OPA/Conftest · Sentinel (advisory / soft / hard) · Terraform Policy (1.17)

Plan

Cloud guardrails

AWS SCP · Azure Policy deny · GCP org policy

Apply

Retired

tfsec (merged into Trivy) · Terrascan (archived Nov 2025)

—

Start with 10 policies you would be embarrassed to violate

Public storage · unencrypted data stores · wildcard IAM · open management ports · missing ownership tags. Run advisory for two sprints, then mandatory.

Secrets: never in code, never in state

```
# Read at apply time; never written to plan or state (TF >= 1.10)
ephemeral "aws_secretsmanager_secret_version" "db" {
  secret_id = var.db_secret_arn
}

resource "aws_db_instance" "main" {
  # ...
  # Write-only argument: sent to the API, never recorded (TF >= 1.11)
  password_wo = (
    ephemeral.aws_secretsmanager_secret_version.db.secret_string
  )
  password_wo_version = 1
}
```

85% of organizations have plaintext secrets in source repositories (Orca, 2025). Terraform state is plaintext too, including every secret it ever managed.



Source of truth: a secrets manager

Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager. Terraform reads at apply time; nothing lives in .tf or .tfvars.



Ephemeral resources and values

Read anew each run, never persisted to plan or state. Ephemeral variables and outputs for pass-through.



Write-only arguments

password_wo and friends on RDS, Secrets Manager, Azure SQL, Cloud SQL, Kubernetes secrets. Bump the _version to rotate.



Dynamic cloud credentials

Vault-backed or OIDC dynamic credentials for the run itself. Zero long-lived secrets in the delivery path.

Compliance automation: preventive and detective



Preventive: stop the change

- Plan-time policy in the pipeline (OPA, Sentinel, tfpolicy)
- Cloud guardrails for anything that bypasses the pipeline: AWS SCPs, Azure Policy deny, GCP org policy
- Secure-by-default modules



Detective: find what slipped

- AWS Config, Azure Policy audit, GCP Security Command Center
- CSPM (Wiz, Prisma, Orca) with code-to-cloud mapping
- Scheduled drift detection and continuous validation



Evidence: for free

- Tag each policy with its control ID: CIS, NIST 800-53, PCI DSS, SOC 2
- Pre-written sets exist: HashiCorp CIS AWS (34 Sentinel policies), NIST 800-53 Rev 5
- Per change: plan + policy verdict + approver + apply log

Why now

Jan 2025

EU DORA applies to financial entities

Mar 2025

PCI DSS 4.0.1 future-dated requirements mandatory

Sep 2026

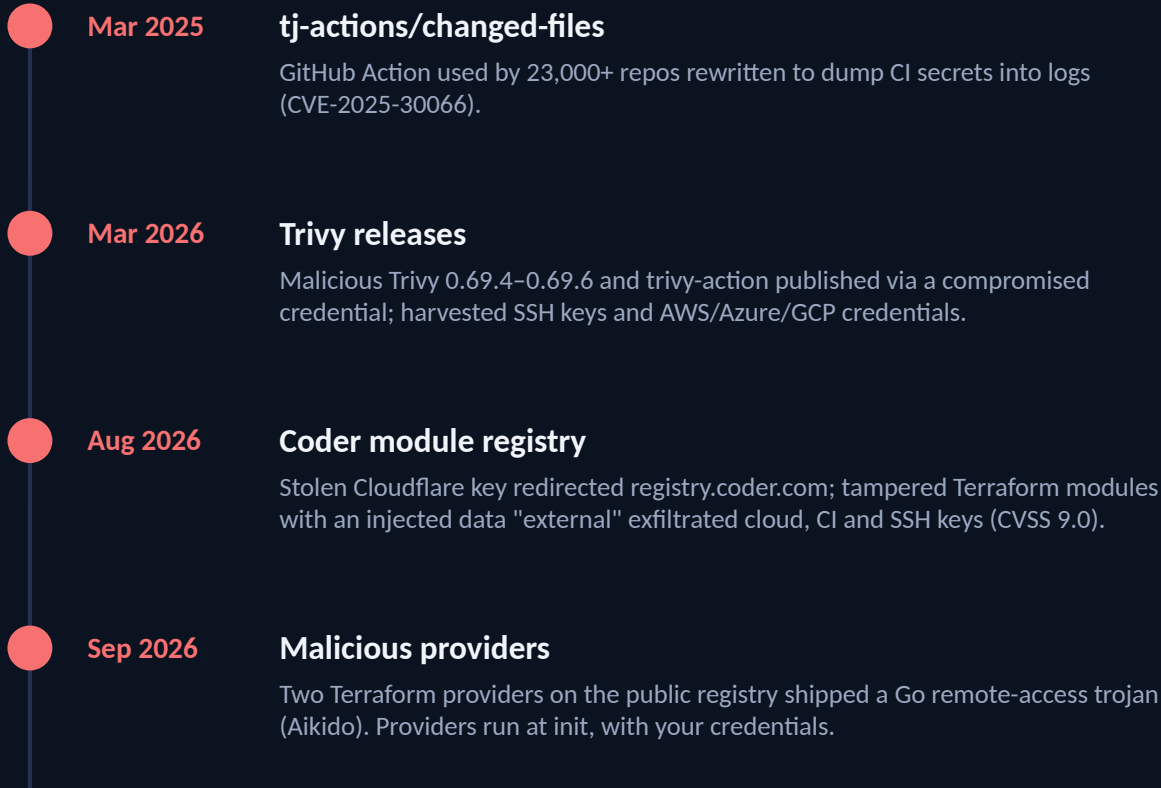
EU Cyber Resilience Act vulnerability reporting duty starts

Dec 2027

CRA full obligations

76% of teams find more compliance issues after deployment than during development (GitLab Global DevSecOps Survey 2025).

Your pipeline is part of the attack surface



Boring, effective defenses

-  Commit `.terraform.lock.hcl` with hashes for every platform (terraform providers `lock -platform=...`)
-  Pin providers, modules and CI actions to versions or commit SHAs, never floating tags
-  Serve from a private mirror with an allow-list (provider_installation network_mirror)
-  Accept only HashiCorp- or partner-signed providers; block unknown sources with policy
-  Ephemeral CI runners: nothing to persist on, nothing to steal later
-  Renovate or Dependabot open upgrade PRs, so pinning never rots

Sources: CISA (2025-03-18); aquasecurity/trivy discussion #10425; GHSA-vx42-ghc9-gw65; The Hacker News (2026-09-23).

Drift detection and continuous validation

48%

make out-of-band changes multiple times a week

35%

had multiple costly production incidents from drift last year

20%

have no drift detection at all

Scheduled plan

plan -detailed-exitcode
or HCP health assessment



Exit code 2

drift detected; JSON diff captured



Alert + ticket

owning team, SLA by environment



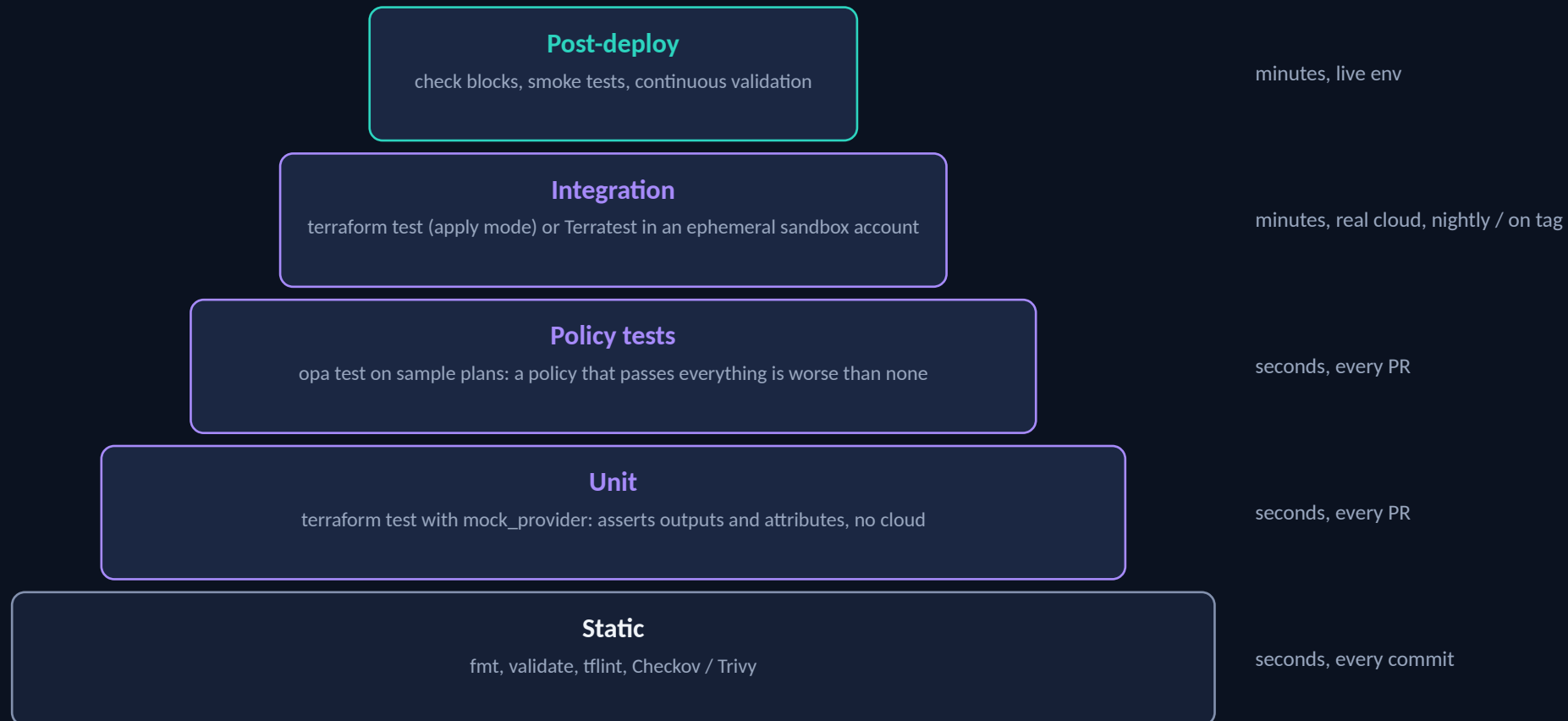
Decide

revert (apply) or codify (import / PR)

- Never auto-apply drift in production without a policy check: a revert can be its own outage
- Continuous validation with check blocks catches what drift cannot: expiring certificates, unhealthy endpoints, quota
- Tools: HCP Terraform health assessments; Spacelift, env0, Scalr, Firefly; open source Atlantis, Digger, Stategraph
- driftctl is in maintenance mode (last release Dec 2023): do not build on it

Firefly, State of IaC 2026 (Aug 2026)

The infrastructure testing pyramid



The rule that keeps it affordable

- Every shared module: unit tests on every PR
- Security-boundary modules only: the integration tier
- Sandbox accounts are ephemeral and auto-destroyed
- Publish JUnit output (terraform test -junit-xml) so CI shows failures inline

Observability and the audit trail



Pipeline events

Plan summary, policy verdict, approver, apply result. Emit as structured events per run.



Platform audit trail

HCP Terraform audit trails are retained 14 days: stream them to Splunk, Datadog or CloudWatch. Atlantis / Spacelift equivalents likewise.



Cloud audit logs

CloudTrail, Azure Activity Log, GCP Audit Logs. Federated session name = run ID ties every API call to a commit and approver.



Five metrics that tell you whether the platform works

Lead time

for an infrastructure change

Policy failure rate

and where in the flow it fails

Drift MTTR

time from detection to resolution

IaC coverage

% of resources under code

Static credentials

pipelines still using them: target 0

LESSONS LEARNED

Lessons from enterprise implementations

1

Governance before scale

Module standards, state layout and the first policies belong in quarter one. Retrofitting across hundreds of workspaces is a migration project.

2

State is production infrastructure

Backups, access reviews, break-glass. The same rigor you give your databases.

3

Contracts beat abstractions

One "universal" module for all clouds ends in a lowest common denominator. Per-cloud modules with a shared interface give consistency without the compromise.

4

Advisory before mandatory

Two sprints of advisory mode, then enforce. Exceptions need an owner and an expiry date, or teams route around the pipeline.

5

Platform and security: one backlog

Only 27% of organizations run them as one function. It is the strongest predictor of whether any of this sticks.

What good looks like: outcomes reported publicly

70%

median reduction in cloud misconfigurations
with IaC security

Snyk, State of Cloud Security 2022

90%

faster deployments, 80% less PR approval time,
zero expired-secret incidents

Riverty case study, HashiCorp 2025

8,000 h

developer hours saved by moving Terraform runs
into pull requests; near-zero direct prod access

Benchling, HashiCorp blog 2026

< 1 day

change lead time and 5% change failure rate for
elite performers: the yardstick

DORA State of DevOps 2024

A 90-day adoption roadmap

Days 1–30

Remove the breach ingredients

- Every state remote, encrypted, locked, versioned
- Static cloud keys in CI replaced with OIDC federation
- Lock files committed; providers and actions pinned
- Static scanning on PRs in advisory mode



Days 31–60

Build the paved road

- First three modules: network, identity, storage, with secure defaults and unit tests
- State split by environment × domain
- Plan and apply roles separated
- Top 10 policies mandatory for production



Days 61–90

Close the loop

- Scheduled drift detection with owners and SLAs
- Pipeline and audit events into the SIEM
- Policies tagged to your compliance framework
- Developer docs and examples; measure the five metrics

Then measure, and iterate: the five metrics on the observability slide are your scorecard.

Key takeaways



One workflow, not one API

Standardize the language, state, plan/apply loop, pipeline and policy layer. Let resources stay cloud-specific.



Security lives in four stages

Author, plan, apply, operate. Each has a control that fits it naturally; none of them belongs at the end.



Kill the long-lived secret

Federated identity for the pipeline, ephemeral values in configuration, encrypted, locked and audited state.



The pipeline is attack surface

Pin, mirror, sign and verify what it runs. Lock files, private mirrors, signed providers, ephemeral runners.

Security stops being the gate at the end and becomes the reason you can ship faster.

Thank you

Perumalsamy Ravindran

Staff Software Engineer



linkedin.com/in/perumalsamyravindran



Sources for every statistic are on the next slide and in the speaker notes.

Conf42 DevSecOps 2026 · 1 October 2026

References

- Flexera, State of the Cloud Report 2025 (Mar 2025) and 2026 (Mar 2026)
- Datadog, State of DevSecOps 2025; State of Cloud Security 2024 and 2025
- Firefly, State of IaC 2025 and 2026 (Aug 2026)
- HashiCorp, Cloud Complexity Report 2025 (Sep 2025)
- Wiz, Cloud Threats Retrospective 2026 (Apr 2026); State of Cloud Risk 2026 (Aug 2026); 38TB disclosure (Sep 2023)
- GitGuardian, State of Secrets Sprawl 2026 (Mar 2026)
- IBM, Cost of a Data Breach Report 2026 (Jul 2026)
- Orca Security, 2025 State of Cloud Security (Jun 2025)
- GitLab, Global DevSecOps Survey 2025 (Nov 2025)
- OCC, Capital One consent order and \$80M penalty (Aug 2020); Capital One 2019 incident facts
- BleepingComputer, Volkswagen / Cariad EV data exposure (Dec 2024)
- CISA alert, tj-actions/changed-files CVE-2025-30066 (Mar 2025)
- aquasecurity/trivy discussion #10425, Trivy supply-chain compromise (Mar 2026)
- GitHub advisory GHSA-vx42-ghc9-gw65, Coder registry compromise (Aug 2026)
- The Hacker News / Aikido, malicious Terraform providers (Sep 2026)
- HashiCorp Terraform releases 1.6–1.17 (github.com/hashicorp/terraform/releases); tfpolicy announcement (Jul 2026)
- HashiCorp docs: sensitive data in state; S3, azurearm, gcs backends; module composition; CLI workspaces; dynamic provider credentials; audit trails
- Google Cloud, Best practices for Terraform (root modules; security)
- AWS Prescriptive Guidance, Terraform AWS Provider best practices: security
- OPA docs, Terraform policy tutorial; Checkov, KICS, Trivy, Terrascan repositories (Sep 2026)
- GitHub docs, OpenID Connect for Actions; Google Cloud, Workload Identity Federation with deployment pipelines
- OpenTofu docs and releases (state encryption, provider for_each); CNCF project page
- EU DORA, NIS2, Cyber Resilience Act; PCI SSC on PCI DSS v4.0.1 dates
- Outcomes: Snyk State of Cloud Security 2022; HashiCorp case studies Riverty (Oct 2025) and Benchling (Feb 2026); DORA Accelerate State of DevOps 2024